PRACTICAL PYTHON / COMPLETE STUDY COURSE

NumPy & pandas

Understand arrays.
Work confidently with tables.
Build analyses you can verify.

30 chapters · 30 exercises with solutions
70 verified Python cells · One complete capstone
Four-week study plan · Quick reference · Two extension projects

Prepared 14 September 2026
Basic Python assumed. All example data are fictional.
Core examples verified on Python 3.12.14, NumPy 2.3.5, and pandas 2.2.3.
Includes documentation-checked pandas 3.0 notes.

YOUR LEARNING PATH

Contents

Read in order for a complete course, or use the chapter links as a reference. Questions appear before their solutions. The companion notebooks preserve executable code and outputs.

01 · How to study this course 3
02 · Setup and the Python bridge 4
03 · Choosing the right data structure 5
04 · Array creation, attributes, and dtypes 6
05 · Indexing, slicing, copies, and views 7
06 · Shapes, axes, and reshaping 8
07 · Broadcasting without guesswork 9
08 · Arithmetic, conditions, and numerical care 10
09 · Statistics, sorting, and unique values 11
10 · Random sampling and linear algebra 12
11 · Series, DataFrames, and a reusable dataset 13
12 · Selection, filtering, and assignment 14
13 · Index alignment and Copy-on-Write 15
14 · Missing values and nullable dtypes 16
15 · Text, categories, duplicates, and schema 17
16 · Transforming columns and building pipelines 18
17 · GroupBy: aggregation, transformation, and filtering 19
18 · Combining tables without multiplying mistakes 20
19 · Reshaping, pivot tables, and MultiIndex 21
20 · Dates, time zones, resampling, and windows 22
21 · Reading and writing data 23
22 · Moving between pandas and NumPy 24
23 · Performance, memory, and scaling 25
24 · Validation, debugging, and clear charts 26
25 · Capstone: an auditable sales analysis 27
26 · Practice bank: 30 exercises 30
27 · Solutions and reasoning 35
28 · Quick reference and common mistakes 41
29 · Concept review with explanations 43
30 · Further projects, mastery checklist, and sources 44

In the PDF, long code lines may wrap for reading. Use the notebook or browser edition to copy complete statements.

STUDY CHAPTER 01

01 · How to study this course

Outcome. By the end, you should be able to manipulate numerical arrays, clean a messy table, combine datasets safely, calculate grouped and time-based metrics, and explain why the results are correct.

This is a practical beginner-to-advanced foundation. It covers the everyday analytical core plus selected advanced topics; it is not an exhaustive catalogue of either library. Specialist topics such as NumPy's C API, distributed computing, and pandas extension-array implementation are outside scope.

Prerequisites: variables, lists, dictionaries, loops, functions, slicing, imports, and basic arithmetic. Chapter 02 provides a short refresher. If these ideas are new, spend a few days practising Python before moving beyond Chapter 04.

Suggested pace: four weeks, about 30–40 hours. The times are planning estimates, not prerequisites for success.

- Week 1, 7–9 hours: Chapters 02–10; predict array shapes before running code. Complete exercises N1–N10.
- Week 2, 8–10 hours: Chapters 11–17; practise selection, missing values, text cleaning, and grouping. Complete exercises P1–P6.
- Week 3, 7–9 hours: Chapters 18–24; work through joins, reshaping, dates, I/O, and performance. Complete exercises P7–P12.
- Week 4, 8–12 hours: Chapters 25–30; complete the capstone without looking at the solution, finish N11–N12 and C1–C6, and review weak areas.

For each topic: read the explanation, predict the output, run the example, change one input, and explain the result aloud. Revisit an exercise after one day and again after a week. Memorising method names is less useful than understanding shape, index alignment, missing values, and the unit represented by each row.

Course files. The browser edition is searchable and printable. The PDF is for offline reading. The study notebook contains the explanations, executed examples, and solutions in reading order. The practice notebook provides 30 unsolved exercises with their own starting data. Work in a copy so you can return to the original.

Reading code. Every Python example assumes `import numpy as np` and `import pandas as pd` from Chapter 02. Within a chapter, run examples from top to bottom. Comments show important expected values; notebook outputs show actual execution. Optional file and plotting recipes are explicitly labelled and are not required for the core examples.

STUDY CHAPTER 02

02 · Setup and the Python bridge

Create a virtual environment for the course. A virtual environment keeps project dependencies separate from other projects. Run these lines in a terminal, not inside a Python cell:

```
python -m venv .venv
# macOS/Linux: source .venv/bin/activate
# Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install numpy pandas jupyterlab matplotlib
python -m jupyter lab
```

Some systems name the executable `python3`; use that in place of `python` when needed. Open a notebook using the environment in which you installed the packages. If an import fails in Jupyter even though installation succeeded, check the selected kernel. Optional Excel and Parquet examples need `openpyxl` and `pyarrow`, respectively.

```
import sys
import numpy as np
import pandas as pd

print("Python:", sys.version.split()[0])
print("NumPy:", np.__version__)
print("pandas:", pd.__version__)

prices = [12, 20, 8]
discounted = [price * 0.9 for price in prices]
record = {"product": "Notebook", "price": 12}

def revenue(quantity, unit_price):
    return quantity * unit_price

print(discounted)
print(record["product"], revenue(3, 12))  # Notebook 36
```

Python slicing uses `start:stop:step`; the stop position is excluded. `range(5)` produces the positions 0 through 4. A dictionary associates keys with values; a pandas column name plays a similar lookup role. Functions let you name a reusable transformation and test it separately.

Version note. The core examples were executed with Python 3.12.14, NumPy 2.3.5, and pandas 2.2.3. Current official documentation was also consulted for NumPy 2.x and pandas 3.0 behavior. The code uses modern conventions, including `default_rng`, `.loc` assignment, and explicit nullable string columns. pandas 3.0 changes are explained where relevant; this is not a claim that the notebook was executed on every version.

STUDY CHAPTER 03

03 · Choosing the right data structure

NumPy supplies an N-dimensional array, `ndarray`, designed for numerical work. A typical numeric array has one dtype for all its elements and uses positions rather than row labels. It is a good fit for vectors, matrices, simulations, and numerical transformations.

pandas supplies `Series` and `DataFrame`. A Series has one dimension plus an index. A DataFrame is a labelled table whose columns can have different dtypes. It is a good fit for spreadsheets, records, CSV files, and business data with identifiers and missing values.

Use pandas to organise and validate data, NumPy for numerical computation where it helps, then restore labels when presenting results. pandas can use NumPy-backed or extension-backed columns; a DataFrame is not simply one homogeneous NumPy array.

```
array = np.array([10, 20, 30])
series = pd.Series([10, 20, 30], index=["A", "B", "C"])
frame = pd.DataFrame({"product": ["A", "B"], "units": [10, 20]})

print([1, 2] * 2)           # list repetition: [1, 2, 1, 2]
print(np.array([1, 2]) * 2) # numeric multiplication: [2, 4]
print(array.shape, series.loc["B"], frame.shape)
```

Before any calculation, ask: What does one element or row represent? Which axis contains observations? Which labels identify matching records? A technically valid operation can still answer the wrong question if the orientation or unit of analysis is wrong.

STUDY CHAPTER 04

04 · Array creation, attributes, and dtypes

Learn five attributes: `ndim` is the number of axes; `shape` is the length of each axis; `size` counts elements; `dtype` describes their representation; `nbytes` reports element-buffer bytes, excluding Python object overhead. For a numeric array, `nbytes = size * itemsize`.

```
a = np.array([[2, 4, 6], [8, 10, 12]], dtype=np.int64)
print(a.ndim, a.shape, a.size, a.dtype, a.nbytes)
# 2 (2, 3) 6 int64 48

print(np.zeros((2, 3)))
print(np.ones((2, 2), dtype=int))
print(np.full((2, 2), 7))
print(np.eye(3))                # identity matrix
print(np.arange(2, 11, 3))      # [2, 5, 8]: stop excluded
print(np.linspace(0, 1, 5))     # [0, .25, .5, .75, 1]
print(np.zeros_like(a))
```

Use `arange` for an integer step and `linspace` when the desired number of evenly spaced points matters. Floating-point steps with `arange` can make endpoint behavior surprising. `np.empty(shape)` allocates without initialising numeric elements: write every element before reading it.

Dtypes are part of correctness. `float64` uses more memory than `float32` but provides more precision. Integer types have finite ranges. Text mixed into a numeric construction can produce a string array rather than numbers. `object` arrays hold Python objects and usually lose much of the performance benefit of homogeneous numeric storage.

```
values = np.array([1.9, 2.1, -3.8])
print(values.astype(np.int64))   # [1, 2, -3]: truncates toward zero
print(np.iinfo(np.int8).min, np.iinfo(np.int8).max)  # -128 127
small = np.array([120], dtype=np.int8)
print(small.astype(np.int64) + 10)  # [130], safe wider arithmetic
print(np.array([1, 2, 3]).astype(float))
```

Do not treat a cast as rounding or validation. Check values before narrowing a dtype. When importing identifiers such as postal codes, preserve them as strings so leading zeros survive.

Reference: NumPy beginner guide.

STUDY CHAPTER 05

05 · Indexing, slicing, copies, and views

An integer selects a position; a slice selects a range. For a two-dimensional array use `a[row, column]`. Negative positions count from the end. A basic slice generally shares storage with its parent. An integer-array or Boolean-array selection creates a copy of the selected values.

```
a = np.arange(12).reshape(3, 4)
print(a)
print(a[1, 2])          # 6
print(a[-1, :])         # [8, 9, 10, 11]
print(a[:, 1:3])        # columns 1 and 2
print(a[::2, ::-1])     # alternate rows, reversed columns
print(a[[0, 2], [1, 3]]) # [1, 11]: paired coordinates
print(a[np.ix_([0, 2], [1, 3])]) # rectangular selection
print(a[a % 3 == 0])    # [0, 3, 6, 9], one-dimensional selection
```

The distinction between paired indexing and rectangular indexing is easy to miss: `a[[0, 2], [1, 3]]` selects two positions, while `np.ix_` builds all four row/column combinations.

```
base = np.array([10, 20, 30, 40])
view = base[1:3]
view[0] = 99
print(base)                    # [10, 99, 30, 40]

independent = base[[1, 2]]
independent[0] = -1
print(base)                    # unchanged
print(np.shares_memory(base, view))         # True
print(np.shares_memory(base, independent))  # False

safe = base[1:3].copy()         # explicit independence
base[base > 30] = 0            # assignment changes the parent
print(base)                    # [10, 0, 30, 0]
```

Reading with advanced indexing returns a copy, but assigning directly through an index updates the target array. `b = a` only creates another reference to the same object. Use `.copy()` when the original must remain independent. These NumPy storage rules are distinct from pandas Copy-on-Write behavior discussed in Chapter 13.

Reference: NumPy copies and views.

STUDY CHAPTER 06

06 · Shapes, axes, and reshaping

For a table-shaped array with shape `(rows, columns)`, axis 0 indexes rows and axis 1 indexes columns. **A reduction removes the named axis.** Therefore `sum(axis=0)` returns a total for each column, while `sum(axis=1)` returns a total for each row. `keepdims=True` leaves the reduced axis with length one, often making later broadcasting easier.

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.sum(axis=0))                 # [5, 7, 9]
print(a.sum(axis=1))                 # [6, 15]
print(a.sum(axis=1, keepdims=True))   # shape (2, 1)
print(a.reshape(3, 2))
print(a.reshape(-1))                 # infer one dimension
print(a.T)                          # transpose: shape (3, 2)

v = np.array([1, 2, 3])
print(v.shape, v.T.shape)            # both (3,)
print(v[:, None].shape)              # column shape (3, 1)
print(v[None, :].shape)              # row shape (1, 3)
```

`reshape` must preserve the number of elements. Only one dimension may be inferred with `-1`. Transposing a one-dimensional array does not turn it into a column. `reshape` can share data or copy, depending on layout; do not assume a copy. `ravel()` returns a flattened view where possible, whereas `flatten()` makes a flattened copy. `squeeze()` removes axes of length one; specifying `axis=` can protect meaningful dimensions.

```
left = np.array([[1, 2], [3, 4]])
right = np.array([[5, 6], [7, 8]])
print(np.concatenate([left, right], axis=0).shape) # (4, 2)
print(np.concatenate([left, right], axis=1).shape) # (2, 4)
print(np.stack([left, right], axis=0).shape)       # (2, 2, 2)
print([part.shape for part in np.array_split(np.arange(7), 3)])
# [(3,), (2,), (2,)]
```

`concatenate` extends an existing axis, so other dimensions must match. `stack` creates a new axis, so inputs must have matching shapes. Repeated concatenation inside a loop repeatedly allocates and copies; collect parts and concatenate once.

STUDY CHAPTER 07

07 · Broadcasting without guesswork

Broadcasting lets compatible shapes participate in elementwise operations without explicitly repeating the smaller input. Compare dimensions from right to left. Each pair must either match or contain a 1; an absent leading dimension behaves as 1. The result uses the matching size, or the size opposite 1.

```
Matrix          (3, 4)
Column offsets     (4,)
Result          (3, 4)

Row offsets     (3, 1)
Matrix          (3, 4)
Result          (3, 4)

Matrix          (3, 4)
Wrong offsets      (3,)   -> incompatible: 4 versus 3
```

```
scores = np.array([[60, 70, 80], [80, 90, 100]], dtype=float)
bonus_by_subject = np.array([5, 0, -5])
bonus_by_student = np.array([2, 4])[:, None]
print(scores + bonus_by_subject)
print(scores + bonus_by_student)

centered = scores - scores.mean(axis=0, keepdims=True)
print(centered)                    # rows [-10, -10, -10], [10, 10, 10]
np.testing.assert_allclose(centered.mean(axis=0), 0)

x = np.array([1, 2, 3])
y = np.array([10, 20])
print(x[:, None] + y[None, :])     # outer addition, shape (3, 2)
```

Two arrays can be broadcast-compatible and still express the wrong computation. On a square matrix, row and column offsets can both appear to fit; write down the intended orientation first. Broadcasting avoids materialising repeated inputs, but the output and intermediate expressions can still be very large. A pairwise-distance calculation for many observations may need batching.

Reference: NumPy broadcasting rules.

STUDY CHAPTER 08

08 · Arithmetic, conditions, and numerical care

Vectorisation expresses a calculation over whole arrays instead of writing a Python loop for every element. Universal functions, or ufuncs, include operations such as `np.sqrt`, `np.exp`, `np.log`, `np.maximum`, and `np.add`. They understand NumPy arrays and broadcasting.

```
price = np.array([100, 80, 50], dtype=float)
quantity = np.array([2, 3, 4])
sales = price * quantity
print(sales)                          # [200, 240, 200]
print(np.sqrt(quantity))
print(np.clip(price, 60, 90))          # [90, 80, 60]

mask = (sales >= 200) & (price < 100)
print(sales[mask])                    # [240, 200]
print(np.where(sales > 200, "high", "regular"))
print(np.any(sales > 230), np.all(quantity > 0))  # True True
```

For elementwise logic use `&`, `|`, and `~`, with parentheses around each comparison. Python's `and` and `or` ask for the truth of a whole object and fail for typical multi-element arrays. Use `.any()` or `.all()` only when you intentionally want one overall Boolean.

Safe division. `np.where(denominator != 0, numerator / denominator, 0)` still evaluates the division expression. A ufunc's `where=` parameter controls which entries it computes. Initialise `out` so skipped entries have known values.

```
numerator = np.array([10., 20., 0.])
denominator = np.array([2., 0., 0.])
ratio = np.divide(
    numerator, denominator,
    out=np.full_like(numerator, np.nan),
    where=denominator != 0,
)
print(ratio)                        # [5, nan, nan]
print(np.isfinite(ratio))            # [True, False, False]
print(np.isclose(0.1 + 0.2, 0.3))    # True
```

Floating-point numbers approximate many decimals. Compare computed values with an appropriate tolerance (`isclose`, `allclose`), not exact equality. Choose tolerances based on the scale and required accuracy of the problem. `isfinite` excludes both NaN and infinity; `isnan` detects NaN only. `np.vectorize` is a convenience wrapper and generally does not turn Python code into a fast compiled numeric kernel.

STUDY CHAPTER 09

09 · Statistics, sorting, and unique values

Statistics describe a population of observations; define that population before choosing an aggregation. A mean is sensitive to extreme values, a median is more resistant, and a quantile is a cutoff within the sorted values. Distinguish missing observations from zero observations.

```
x = np.array([2., 4., 6., 8.])
print(x.sum(), x.mean(), np.median(x))  # 20, 5, 5
print(x.var(ddof=0))                    # 5: divisor n
print(x.var(ddof=1))                    # 6.666...: divisor n - 1
print(np.quantile(x, [0.25, 0.5, 0.75])) # [3.5, 5, 6.5]
print(np.average([80, 90], weights=[1, 3])) # 87.5
print(np.cumsum(x))                     # [2, 6, 12, 20]
print(np.diff(x))                       # [2, 2, 2]

dirty = np.array([2., np.nan, 6.])
print(dirty.mean(), np.nanmean(dirty)) # nan, 4
```

NumPy's default `std` and `var` use `ddof=0`; pandas Series uses `ddof=1` by default. Pass `ddof` explicitly when comparing libraries. A `nan`-aware function ignores NaNs, but an all-missing slice has no valid mean. Zero is not a neutral replacement for an unknown measurement.

```
x = np.array([40, 10, 30, 10])
print(np.sort(x))                    # [10, 10, 30, 40], sorted copy
print(np.argsort(x, kind="stable")) # positions [1, 3, 2, 0]
print(x.argmax())                    # 0: position of first maximum
values, counts = np.unique(x, return_counts=True)
print(values, counts)                # [10, 30, 40], [2, 1, 1]
print(np.isin(x, [10, 30]))
```

`np.sort(x)` returns a sorted copy; `x.sort()` sorts in place. `argmax` and `argmin` return positions, not values. Stable sorting preserves input order among equal keys. For large top-k selections, `argpartition` can avoid a full sort, but the selected items still need sorting if order matters.

STUDY CHAPTER 10

10 · Random sampling and linear algebra

Use a local random generator. A seed makes a run reproducible within the relevant environment and sequence of calls. It does not make a sample representative, and bit-for-bit reproducibility across all future library versions is not guaranteed. Record package versions for reproducible studies.

```
rng = np.random.default_rng(42)
print(rng.integers(1, 7, size=8))   # die faces; upper bound excluded
print(rng.normal(loc=0, scale=1, size=4))
print(rng.choice(["A", "B", "C"], size=2, replace=False))

measurements = np.array([10., 12., 14., 16.])
bootstrap_rng = np.random.default_rng(7)
samples = bootstrap_rng.choice(measurements, size=(2000, 4), replace=True)
bootstrap_means = samples.mean(axis=1)
interval = np.quantile(bootstrap_means, [0.025, 0.975])
print("Illustrative bootstrap percentile interval:", interval)
```

The bootstrap example resamples observations with replacement. Its interval is illustrative, particularly with only four observations. Meaningful inference requires suitable sampling assumptions; dependent time-series observations require more careful resampling.

Reference: NumPy Generator API and reproducibility note.

For two-dimensional matrices, `A @ B` performs matrix multiplication. `A * B` multiplies matching elements. If `A` has shape `(m, k)` and `B` has shape `(k, n)`, `A @ B` has shape `(m, n)`.

```
A = np.array([[3., 1.], [1., 2.]])
b = np.array([9., 8.])
solution = np.linalg.solve(A, b)
print(solution)                     # [2, 3]
np.testing.assert_allclose(A @ solution, b)
print(A * A)
print(A @ A)
print(np.linalg.norm(solution))

design = np.column_stack([np.ones(4), np.arange(4)])
target = np.array([1., 3., 5., 7.])
coef, residuals, rank, singular_values = np.linalg.lstsq(
    design, target, rcond=None
)
print(coef)                         # intercept 1, slope 2
```

Use `solve(A, b)` for a nonsingular square system rather than calculating `inv(A) @ b`. Use `lstsq` for least squares. A singular system has no unique solution; an ill-conditioned system can amplify small input errors. `np.linalg.cond(A)` helps diagnose conditioning. For real symmetric matrices, `eigh` provides eigenvalues and eigenvectors; SVD (`np.linalg.svd`) underlies many dimensionality-reduction methods.

STUDY CHAPTER 11

11 · Series, DataFrames, and a reusable dataset

In this dataset, **one row represents one order** with one product. `order_id` is unique. `customer_id` can repeat because a customer can place several orders. Revenue is quantity multiplied by unit price. Real order-line data can have several rows per order, in which case counting rows is not the same as counting orders.

```
def make_sales():
    return pd.DataFrame({
        "order_id": [101, 102, 103, 104, 105, 106],
        "customer_id": [1, 2, 1, 3, 2, 4],
        "date": pd.to_datetime([
            "2026-01-02", "2026-01-03", "2026-01-03",
            "2026-02-01", "2026-02-02", "2026-02-03",
        ]),
        "region": pd.Series(
            ["North", "South", "North", "West", "South", "West"],
            dtype="string",
        ),
        "product": pd.Series(
            ["Pen", "Book", "Book", "Pen", "Pen", "Book"],
            dtype="string",
        ),
        "qty": [2, 1, 3, 4, 2, 1],
        "price": [10., 30., 30., 10., 10., 30.],
    }).assign(revenue=lambda d: d["qty"] * d["price"])

sales = make_sales()
print(sales.to_string(index=False))
print(sales["revenue"].sum())       # 230
```

`pd.DataFrame` can be built from column dictionaries, lists of row dictionaries, arrays, or imported files. Each column has one dtype, but different columns can use different dtypes. The default row index is a `RangeIndex`; it is a label system, not necessarily a business identifier.

```
sales = make_sales()
print(sales.head(3))
print(sales.shape)                  # (6, 8)
print(sales.columns.tolist())
print(sales.dtypes)
print(sales[["qty", "price", "revenue"]].describe())
print(sales["region"].value_counts(dropna=False))
```

`head` and `tail` show a sample; `info()` prints dtypes and non-null counts; `describe()` summarises distributions. Also inspect `isna().sum()`, `duplicated().sum()`, and key uniqueness. A few plausible first rows do not establish that the whole dataset is valid.

Reference: pandas introductory walkthrough.

STUDY CHAPTER 12

12 · Selection, filtering, and assignment

`df["col"]` returns a Series. `df[["col"]]` returns a DataFrame. `.loc` selects by labels; `.iloc` selects by integer positions. Use `.at[row_label, column_label]` and `.iat[row_position, column_position]` for a single scalar when useful.

```
sales = make_sales().set_index("order_id")
print(sales.loc[103, "revenue"])         # 90
print(sales.iloc[2, -1])                 # 90
print(sales.loc[102:104, ["product", "qty"]]) # stop label included
print(sales.iloc[1:4, :2])               # stop position excluded

mask = (sales["region"] == "North") & (sales["qty"] >= 2)
print(sales.loc[mask, ["product", "revenue"]])
print(sales.loc[sales["product"].isin(["Book"])])
print(sales.loc[sales["revenue"].between(25, 50)])

sales.loc[sales["region"] == "West", "price"] = 12.0
sales["revenue"] = sales["qty"] * sales["price"]
print(sales.loc[[104, 106], ["price", "revenue"]])
```

Label slicing includes both endpoints when labels and ordering support that slice. Position slicing follows normal Python's excluded stop. Sorting an index is often necessary for predictable range queries, especially with dates and MultiIndexes. A mask should be built from the same DataFrame so its index matches the target.

Use one assignment operation such as `df.loc[mask, "price"] = value`. Do not write `df[mask]["price"] = value`. When recalculating a derived column, remember that pandas does not automatically maintain a spreadsheet-style formula linking it to its source columns.

STUDY CHAPTER 13

13 · Index alignment and Copy-on-Write

Alignment is central to pandas. Arithmetic between Series matches index labels, not their physical order. Assigning a Series to a column also aligns by labels. Missing matches generally introduce missing values.

```
left = pd.Series([10, 20], index=["A", "B"])
right = pd.Series([1, 2], index=["B", "C"])
print(left + right)                # A NaN, B 21, C NaN
print(left.add(right, fill_value=0)) # A 10, B 21, C 2

frame = pd.DataFrame({"x": [100, 200]}, index=["A", "B"])
frame["aligned"] = pd.Series([1, 2], index=["B", "A"])
frame["positional"] = np.array([1, 2])
print(frame)                       # aligned [2, 1]; positional [1, 2]
```

Choosing positional assignment is reasonable only after checking row order and length. Removing labels can silence an error while creating an incorrect match. `reset_index()` makes the old index a column; `reset_index(drop=True)` discards it. `set_index()` selects labels from columns; `reindex()` conforms data to a requested set of labels and can insert missing rows.

pandas 3.0 uses Copy-on-Write as its default and only mode. A derived Series or DataFrame behaves independently when mutated through pandas operations. Chained assignment never updates the intended parent under this model. Write directly to the object you want to change using `.loc`, or assign a complete transformed column. In pandas 2.x, behavior depends on the Copy-on-Write option; `.loc` remains the robust idiom.

```
original = pd.DataFrame({"x": [1, 2, 3]})
independent = original.copy()
independent.loc[0, "x"] = 99
print(original["x"].tolist())       # [1, 2, 3]
print(independent["x"].tolist())    # [99, 2, 3]
```

Do not use `df["x"].fillna(0, inplace=True)` expecting a guaranteed parent update; use `df["x"] = df["x"].fillna(0)`. `inplace=True` does not guarantee faster execution or no allocation. Even a deep pandas copy does not recursively clone mutable Python objects stored inside object columns.

Reference: pandas Copy-on-Write guide.

STUDY CHAPTER 14

14 · Missing values and nullable dtypes

Missing data can mean unknown, not collected, invalid, or not applicable. Identify which meaning applies before deleting or imputing. pandas uses representations including `np.nan`, `pd.NA`, and `NaT`, depending on dtype. Always detect missingness with `isna()` or `notna()`.

```
data = pd.DataFrame({
    "name": pd.Series(["Mira", None, "Dev"], dtype="string"),
    "age": pd.Series([20, None, 30], dtype="Int64"),
    "score": [80., np.nan, 100.],
})
print(data.isna().sum())
clean = data.assign(
    name=data["name"].fillna("Unknown"),
    score=data["score"].fillna(data["score"].median()),
)
print(clean)
print(data.dropna(subset=["age"]))
print(data["score"].mean())          # 90: skips missing values
print(pd.Series([np.nan]).sum(min_count=1))  # nan, not zero
```

The capital `I` in `Int64` selects pandas' nullable integer dtype; NumPy's `int64` cannot directly represent missing integers. Nullable `boolean` supports unknown truth values. Comparisons involving `pd.NA` may remain unknown; the truth value of `pd.NA` is ambiguous. Make your missing-value policy explicit before filtering.

For numeric parsing, `pd.to_numeric(s, errors="coerce")` turns invalid input into missing values. Count these conversions and inspect their causes. For date parsing use `pd.to_datetime`. `convert_dtypes()` can infer nullable dtypes, but you should still review whether the resulting schema matches the meaning of each field.

```
raw = pd.Series(["10", "bad", "30"], dtype="string")
parsed = pd.to_numeric(raw, errors="coerce")
invalid = raw.notna() & parsed.isna()
print(parsed.tolist(), invalid.sum())  # one invalid value
```

Median filling is a teaching example, not an automatic cleaning rule. In predictive modelling, learn imputation parameters from training data only. Forward filling is appropriate only if carrying the last known value forward has a defensible meaning, and it should be performed within each relevant entity after sorting by time.

Reference: pandas missing-data guide.

STUDY CHAPTER 15

15 · Text, categories, duplicates, and schema

Text operations are available through `.str`. Normalise only when the normalisation is valid: trimming spaces is usually sensible, but lowercasing a case-sensitive identifier could merge distinct records.

```
text = pd.Series(["  NORTH ", "south", None, "West-01"], dtype="string")
normal = text.str.strip().str.lower()
print(normal.tolist())
print(normal.str.contains("north", regex=False, na=False).tolist())
print(normal.str.replace("-", "_", regex=False).tolist())
print(normal.str.extract(r"([0-9]+)", expand=False).tolist())

raw = pd.DataFrame({"id": [1, 1, 2], "value": [10, 10, 20]})
print(raw.duplicated().tolist())    # [False, True, False]
deduplicated = raw.drop_duplicates()
assert deduplicated["id"].is_unique
```

`duplicated()` examines complete rows by default; `duplicated(subset=["id"])` examines keys. A repeated key can indicate a valid one-to-many relationship, a duplicate export, or conflicting records. Investigate before retaining the first or last occurrence. If choosing the newest record, sort by a reliable timestamp first and define how ties are resolved.

For a small repeated vocabulary, a categorical dtype can reduce memory and encode permitted or ordered values. Its benefit depends on cardinality; near-unique identifiers often gain little. A value outside a specified category set can become missing when cast, so validate first.

```
level_type = pd.CategoricalDtype(
    categories=["low", "medium", "high"], ordered=True
)
levels = pd.Series(["high", "low", "medium"], dtype=level_type)
print(levels.sort_values().tolist())  # low, medium, high
```

In pandas 3.0, inferred string data uses a dedicated `str` dtype by default. The explicit nullable `dtype="string"` used in this course retains `pd.NA` semantics across pandas 2.x and 3.0. Do not assume every text column has `object` dtype or hard-code the storage backend. Inspect dtypes and use `isna()` for missingness.

Reference: pandas 3.0 string migration guide.

STUDY CHAPTER 16

16 · Transforming columns and building pipelines

Start with direct vectorised expressions. Use `map` for a lookup on a Series, `replace` for value substitution, `assign` for derived columns, and `pipe` to name a DataFrame-to-DataFrame transformation. `apply` is flexible but often slower and less predictable in output shape than dedicated methods.

```
sales = make_sales()
region_code = {"North": "N", "South": "S", "West": "W"}
result = (
    sales
    .assign(
        region_code=lambda d: d["region"].map(region_code),
        large_order=lambda d: d["revenue"] >= 50,
        discounted_revenue=lambda d: d["revenue"] * 0.9,
    )
    .sort_values(["revenue", "order_id"], ascending=[False, True])
    .reset_index(drop=True)
)
print(result[["order_id", "region_code", "large_order"]])

bands = pd.cut(
    sales["revenue"], bins=[0, 25, 50, np.inf],
    labels=["small", "medium", "large"], right=True,
)
print(bands.tolist())
```

`map(dict)` produces missing values for unmapped keys; `replace(dict)` leaves unmapped values unchanged. `cut` uses numeric boundaries; `qcut` aims for groups with similar numbers of observations, but ties may prevent the requested number of distinct bins. Learn cut points from training data when bins are used in a predictive pipeline.

```
def add_revenue(frame):
    return frame.assign(revenue=frame["qty"] * frame["price"])

pipeline_result = make_sales().drop(columns="revenue").pipe(add_revenue)
assert pipeline_result["revenue"].sum() == 230
```

Long chains should express a coherent stage of work. Name intermediate results when doing so makes validation easier. A concise expression is not an improvement if nobody can check what it means. Prefer bracket column access over attributes: `df["count"]` is a column lookup, whereas `df.count` is a method.

STUDY CHAPTER 17

17 · GroupBy: aggregation, transformation, and filtering

Grouping partitions rows by keys. An **aggregation** returns summaries per group. A **transformation** returns values aligned to the original rows. A **filter** keeps or removes whole groups. Choose the desired output shape before choosing a method.

```
sales = make_sales()
summary = sales.groupby("region", as_index=False, dropna=False).agg(
    orders=("order_id", "nunique"),
    units=("qty", "sum"),
    revenue=("revenue", "sum"),
    mean_order=("revenue", "mean"),
)
print(summary)
# North: 2 orders, 5 units, 110 revenue, mean 55
# South: 2 orders, 3 units, 50 revenue, mean 25
# West:  2 orders, 5 units, 70 revenue, mean 35

region_total = sales.groupby("region")["revenue"].transform("sum")
sales["region_share"] = sales["revenue"] / region_total
print(sales[["order_id", "region", "region_share"]])
np.testing.assert_allclose(
    sales.groupby("region")["region_share"].sum().to_numpy(), 1
)

kept = sales.groupby("region").filter(lambda g: g["revenue"].sum() >= 70)
print(sorted(kept["region"].unique()))  # North, West
```

`size()` counts rows; `count()` counts non-missing values separately per column; `nunique()` counts distinct values. `groupby` normally drops missing keys; use `dropna=False` when they should form a group. For categorical keys, specify `observed=True` for observed combinations or `observed=False` when you deliberately want category combinations represented; this default changed across pandas versions.

Weighted averages require weights. Average unit selling price is `sum(revenue) / sum(qty)`, not an unweighted mean of each row's unit price. Similarly, aggregate rates by aggregating the numerator and denominator, unless an unweighted mean of subgroup rates is explicitly the question.

Prefer named aggregations and `transform` to a custom `groupby.apply` when possible. `apply` is useful for genuinely custom group logic, but its result index and inclusion of grouping columns are version-sensitive and need careful checking.

Reference: pandas GroupBy guide.

STUDY CHAPTER 18

18 · Combining tables without multiplying mistakes

Use `merge` to match records on keys and `concat` to append compatible tables or align them side by side. Join types answer different questions: inner retains matching keys, left preserves the left table's keys, outer retains keys from both, and right preserves the right table's keys.

```
sales = make_sales()
customers = pd.DataFrame({
    "customer_id": [1, 2, 3, 4],
    "segment": ["Retail", "Business", "Retail", "Business"],
})
joined = sales.merge(
    customers, on="customer_id", how="left",
    validate="many_to_one", indicator=True,
)
assert len(joined) == len(sales)
assert joined["_merge"].eq("both").all()
assert joined["revenue"].sum() == 230
print(joined[["order_id", "segment", "_merge"]])

combined = pd.concat([sales.iloc[:3], sales.iloc[3:]], ignore_index=True)
assert len(combined) == 6
```

`validate="many_to_one"` means the right-hand key must be unique; many left rows may share it. Use `one_to_one`, `one_to_many`, or `many_to_many` to express the intended relationship, but note that `many_to_many` allows duplication rather than protecting against it. `indicator=True` identifies unmatched rows. Use `suffixes=` when both inputs contain non-key columns with the same name.

If a key occurs twice on the left and three times on the right, an equality join creates six rows for that key. Totals may increase even though no exception is raised. Validate key uniqueness, row count, unmatched keys, and additive totals before trusting the result.

pandas can match null join keys to one another, unlike typical SQL equality joins. Decide how missing identifiers should be treated before merging. `concat(axis=1)` aligns by index, and `concat(axis=0)` aligns by column name. Neither operation automatically performs a business-key match. Avoid the removed `DataFrame.append`; collect frames and use `pd.concat`.

Reference: pandas merging and concatenation guide.

STUDY CHAPTER 19

19 · Reshaping, pivot tables, and MultiIndex

In long data, a measured variable can occupy one column with an accompanying category column. In wide data, categories become separate columns. Choose the layout that makes the next operation simple. Long form usually works well for grouping and plotting; wide form can be useful for comparisons and matrix operations.

```
sales = make_sales()
wide = sales.pivot_table(
    index="region", columns="product", values="revenue",
    aggfunc="sum", fill_value=0, observed=True,
)
print(wide)
#          Book  Pen
# North      90   20
# South      30   20
# West       30   40

long = wide.reset_index().melt(
    id_vars="region", var_name="product", value_name="revenue"
)
print(long)
assert long["revenue"].sum() == sales["revenue"].sum()

grouped = sales.groupby(["region", "product"])["revenue"].sum()
print(grouped.loc[("North", "Book")]) # 90
print(grouped.unstack("product", fill_value=0))
```

`pivot` requires a unique value for each index/column pair; it does not aggregate duplicate pairs. `pivot_table` aggregates, and its default aggregation is a mean, so specify `aggfunc` explicitly for totals. `melt` converts selected columns into variable/value rows. `stack` and `unstack` move labels between column and index levels. `crosstab` is convenient for frequency counts.

A MultiIndex contains multiple label levels. It often arises from grouping on several keys. `reset_index()` can turn the result back into ordinary columns, and `.xs(key, level=...)` can select across a named level. Sorting a MultiIndex improves predictable slicing. Use meaningful level names rather than relying on level positions.

STUDY CHAPTER 20

20 · Dates, time zones, resampling, and windows

Convert strings into datetime values before calendar calculations. Use an explicit format when input conventions are known, especially if day and month can be confused. `errors="coerce"` makes invalid dates `NaT`; count and inspect them. Date attributes are available through `.dt` on a datetime Series.

```
sales = make_sales()
sales["month"] = sales["date"].dt.to_period("M")
print(sales.groupby("month")["revenue"].sum()) # Jan 140, Feb 90

monthly = (
    sales.set_index("date").sort_index()["revenue"]
    .resample("MS").sum(min_count=1)
)
print(monthly)                      # month-start labels
print(sales["date"].dt.day_name().tolist())

daily = pd.Series(
    [10., 20., 30., 40.],
    index=pd.date_range("2026-01-01", periods=4, freq="D"),
)
print(daily.rolling(3, min_periods=1).mean().tolist()) # 10, 15, 20, 30
print(daily.shift(1).tolist())      # nan, 10, 20, 30
print(daily.pct_change(fill_method=None).tolist()) # fractions, not percent
```

`resample` groups by time bins; `asfreq` conforms to a frequency without aggregating. `rolling(3)` means three observations, while `rolling("3D")` means a time interval on a datetime index and can contain a variable number of observations. `expanding()` uses all data seen so far. Sort before windows or lag calculations. For multiple entities, group first so values do not leak across entities.

`pct_change` produces fractional change: multiply by 100 for percentage points on a percent display. A zero previous value makes the rate undefined or infinite, so define a denominator policy.

```
utc_times = pd.to_datetime(["2026-01-01T00:00:00Z"], utc=True)
print(utc_times.tz_convert("Asia/Kolkata")) # 05:30 at +05:30

local_naive = pd.DatetimeIndex(["2026-01-01 09:00"])
aware = local_naive.tz_localize("Asia/Kolkata")
print(aware.tz_convert("UTC"))             # 03:30 UTC
```

`tz_localize` attaches the meaning of a local time zone to naive clock times; `tz_convert` changes the displayed zone of an already aware timestamp. Daylight-saving transitions can create ambiguous or nonexistent local times, requiring explicit handling. `utc=True` on a naive input assumes that input represents UTC; do not use it to reinterpret local clock times accidentally.

Reference: pandas time-series guide.

STUDY CHAPTER 21

21 · Reading and writing data

CSV is widely supported and human-readable, but does not preserve a rich schema automatically. Specify important dtypes during import. Parquet preserves many dtypes and is efficient for column-oriented analytics. Excel is useful for spreadsheet exchange. SQL lets a database filter and aggregate data before it reaches Python.

This in-memory CSV example runs without external files:

```
from io import StringIO

csv_text = "customer_id,joined,amount\n001,2026-01-01,10.5\n002,2026-01-02,20\n"
imported = pd.read_csv(
    StringIO(csv_text), dtype={"customer_id": "string"},
    parse_dates=["joined"],
)
print(imported)
assert imported.loc[0, "customer_id"] == "001"
buffer = StringIO()
imported.to_csv(buffer, index=False)
print(buffer.getvalue())
```

Optional local-file recipes. Run these only after supplying the named files and installing any required engine. These recipes are references, not cells in the executed core examples.

```
pd.read_csv("orders.csv", usecols=["order_id", "qty"], dtype={"qty": "Int64"})
pd.read_csv("orders.csv", na_values=["MISSING"], encoding="utf-8")
pd.read_excel("orders.xlsx", sheet_name="Orders", engine="openpyxl")
pd.read_parquet("orders.parquet")
sales.to_csv("clean_orders.csv", index=False)
sales.to_excel("clean_orders.xlsx", index=False)
sales.to_parquet("clean_orders.parquet", index=False)
np.save("features.npy", numeric_array)
restored = np.load("features.npy", allow_pickle=False)
np.savez_compressed("arrays.npz", train=train_array, test=test_array)
```

CSV's default missing markers may conflict with valid codes such as `NA`. Review `keep_default_na` and `na_values` for the dataset. Malformed dates may need a separate explicit `to_datetime` conversion and validation. Export with `index=False` when the row index is not part of the data contract.

For SQL, use parameterised queries supported by the database connector rather than string interpolation of input values. A typical call is `pd.read_sql_query(query, connection, params=parameters)`. Push appropriate selection, joins, and aggregation to the database when reading a large table.

Chunking reduces peak memory for operations that can be combined correctly:

```
chunk_text = "qty,price\n2,10\n1,30\n3,30\n"
total = 0.0
for chunk in pd.read_csv(StringIO(chunk_text), chunksize=2):
    total += (chunk["qty"] * chunk["price"]).sum()
print(total)                          # 140
```

Sums and counts combine naturally across chunks; a global mean needs total sum divided by total count. Averaging chunk means is incorrect when chunk sizes differ. Global deduplication, medians, and some joins need additional state or a different processing approach.

STUDY CHAPTER 22

22 · Moving between pandas and NumPy

Use `.to_numpy()` to move values into NumPy deliberately. Labels disappear, heterogeneous columns may force a common dtype, and nullable values may need conversion. Keep the index and feature names if you will attach results back to records.

```
sales = make_sales()
features = ["qty", "price", "revenue"]
X = sales[features].to_numpy(dtype=float, copy=True)
means = X.mean(axis=0, keepdims=True)
scales = X.std(axis=0, ddof=0, keepdims=True)
Z = np.divide(X - means, scales, out=np.zeros_like(X), where=scales != 0)
standardised = pd.DataFrame(Z, index=sales.index, columns=features)
print(standardised.round(3))
np.testing.assert_allclose(standardised.mean().to_numpy(), 0, atol=1e-12)

nullable = pd.Series([1, None, 3], dtype="Int64")
print(nullable.to_numpy(dtype=float, na_value=np.nan))
```

Standardisation subtracts a feature's mean and divides by its standard deviation. This example assigns zero to constant features by policy; another application may drop those features. For machine learning, split train/test first, estimate means and scales on training data, and reuse those same parameters for validation and test data.

Under pandas Copy-on-Write, `.to_numpy()` can expose read-only shared storage. `copy=True` requests an independent array when you intend to mutate values in NumPy. Do not force writeability on a shared array to bypass pandas' mutation rules. After conversion, preserve both feature order and row order.

STUDY CHAPTER 23

23 · Performance, memory, and scaling

Performance comes from doing less work, using the right representation, and avoiding unnecessary Python-level iteration. Measure before optimising. A fast implementation of the wrong calculation is still wrong.

- Read only needed columns and rows. Choose suitable dtypes after checking value ranges.
- Prefer vectorised numeric operations, built-in reductions, and dedicated GroupBy methods.
- Build a list of frames or arrays and concatenate once rather than growing a structure repeatedly.
- Avoid row-wise `apply(axis=1)` for expressions that can operate directly on columns. If iteration is necessary, `itertuples(index=False)` usually has less overhead than `iterrows()`.
- Use categorical storage for genuinely repeated categories; inspect memory rather than assuming a saving.
- Avoid converting everything to `object`. Use native numeric, datetime, string, or appropriate extension dtypes.
- Check broadcasting intermediates: pairwise operations can scale with the product of input sizes.

```
sales = make_sales()
print(sales.memory_usage(deep=True))
print("Total DataFrame bytes:", sales.memory_usage(deep=True).sum())
matrix = np.zeros((1000, 100), dtype=np.float64)
print("Array buffer bytes:", matrix.nbytes)  # 800000
```

A numeric array with one million rows, 100 columns, and 8-byte values needs about 800 million bytes for its element buffer alone. Intermediate arrays, indices, object storage, and temporary copies add to peak memory. A file's compressed size is not its in-memory size.

In Jupyter, `%timeit expression` helps compare repeated execution time, but results depend on data size, hardware, data layout, and warm-up effects. Confirm results are equivalent before comparing speed. For data that cannot fit comfortably in memory, consider database aggregation, appropriate chunking, or a specialised out-of-core system. pandas operations are primarily designed for in-memory analysis.

STUDY CHAPTER 24

24 · Validation, debugging, and clear charts

Build checks around the meaning of the data. Useful invariants include unique keys, nonnegative quantities when returns are excluded, complete required fields, expected categories, preserved row counts after enrichment, and conserved totals after reshaping.

```
sales = make_sales()
assert sales["order_id"].is_unique
assert sales[["order_id", "date", "qty", "price"]].notna().all().all()
assert sales["qty"].gt(0).all()
assert sales["region"].isin(["North", "South", "West"]).all()
np.testing.assert_allclose(sales["revenue"], sales["qty"] * sales["price"])

expected = pd.Series(
    [110., 50., 70.], index=pd.Index(["North", "South", "West"], name="region"),
    name="revenue",
)
actual = sales.groupby("region")["revenue"].sum()
pd.testing.assert_series_equal(actual, expected, check_index_type=False)
```

Assertions are convenient for learning and tests. In a production validation pipeline, use explicit checks and exceptions or a validation system, because Python can disable `assert` statements in optimised mode. Do not use `check_dtype=False` merely to hide a schema error; decide whether dtype differences are meaningful.

When something is wrong, inspect `shape`, `dtypes`, `head`, key uniqueness, missing counts, and index labels. Reduce the problem to a few rows. For a broadcasting error, write shapes from right to left. For an unexpectedly large joined table, count duplicates on each join key. For unexpected NaNs, check alignment and parsing before filling them.

Choose a chart that answers the question. Use a bar chart for category totals, a line chart for ordered time observations, a histogram for a distribution, and a scatterplot for a relationship. Label units, identify the aggregation and period, and explain whether missing periods represent zero or unavailable data.

Optional plotting recipe; requires Matplotlib:

```
import matplotlib.pyplot as plt
monthly = make_sales().set_index("date")["revenue"].resample("MS").sum()
ax = monthly.plot(marker="o", title="Monthly order revenue, Jan–Feb 2026")
ax.set_xlabel("Month")
ax.set_ylabel("Revenue (currency units)")
plt.tight_layout()
plt.show()
```

Two monthly observations can show their difference but do not establish a stable trend or seasonality. A correlation describes association, not a causal effect. Always connect the visual claim to what the data can support.

STUDY CHAPTER 25

25 · Capstone: an auditable sales analysis

Your task. Clean a fictional order export, enrich it with a customer table, calculate revenue by region and month, and explain what was excluded. Work from the raw rows below. Attempt the workflow before reading the solution.

Business rules. One row is one order. Remove exact duplicate export rows. Quarantine rows with missing or invalid required fields: order ID, customer ID, date, quantity, or price. IDs and quantities must be positive whole numbers; price must be nonnegative. Allow only North, South, and West regions and Book or Pen products. Unmatched customer IDs remain in the sales analysis as `Unknown` and are reported separately. Do not invent quantities or prices. This dataset contains no returns, refunds, taxes, or shipping charges.

Deliverables: a clean enriched table, rejected rows with reasons, a data-quality reconciliation, a regional revenue summary, monthly revenue, average order value, a NumPy feature matrix, and a short conclusion.

```
raw_orders = pd.DataFrame({
    "order_id": [201, 202, 202, 203, 204, 205, 206, 207, 208],
    "customer_id": [1, 2, 2, 1, 3, 2, 4, 4, 99],
    "date": ["2026-01-01", "2026-01-02", "2026-01-02", "2026-01-03",
             "2026-02-01", "2026-02-02", "2026-02-03", "bad-date", "2026-02-04"],
    "region": [" north ", "south", "south", "NORTH", "west", "south", "west", "west", "west"],
    "product": ["Pen", "Book", "Book", "Book", "Pen", "Pen", "Book", "Book", "Pen"],
    "qty": ["2", "1", "1", "3", "4", None, "1", "1", "2"],
    "price": ["10", "30", "30", "30", "10", "10", "bad", "30", "10"],
})
customer_lookup = pd.DataFrame({
    "customer_id": [1, 2, 3, 4],
    "segment": ["Retail", "Business", "Retail", "Business"],
})
print(raw_orders.to_string(index=False))
```

Try first: predict the accepted order IDs and total revenue on paper. The main challenge is deciding what constitutes valid data, not writing the multiplication.

Solution: preserve, parse, validate

Keep the raw table unchanged. Drop exact duplicates before normalisation so the reconciliation clearly identifies repeated export rows. Parsing with coercion allows invalid fields to be flagged without stopping at the first error. The original values remain available in `raw_orders` for investigation.

```
duplicate_mask = raw_orders.duplicated()
parsed = raw_orders.loc[~duplicate_mask].copy()
parsed["region"] = parsed["region"].astype("string").str.strip().str.title()
parsed["product"] = parsed["product"].astype("string").str.strip().str.title()
parsed["date"] = pd.to_datetime(parsed["date"], format="%Y-%m-%d", errors="coerce")
for column in ["order_id", "customer_id", "qty", "price"]:
    parsed[column] = pd.to_numeric(parsed[column], errors="coerce")

checks = pd.DataFrame(index=parsed.index)
for column in ["order_id", "customer_id", "qty"]:
    checks["invalid_" + column] = ~(
        parsed[column].notna() & np.isfinite(parsed[column])
        & parsed[column].gt(0) & parsed[column].mod(1).eq(0)
    )
checks["invalid_date"] = parsed["date"].isna()
checks["invalid_price"] = ~(
    parsed["price"].notna() & np.isfinite(parsed["price"]) & parsed["price"].ge(0)
)
checks["invalid_region"] = ~parsed["region"].isin(["North", "South", "West"])
checks["invalid_product"] = ~parsed["product"].isin(["Pen", "Book"])
bad_row = checks.any(axis=1)

rejected = parsed.loc[bad_row].copy()
rejected["reason"] = checks.loc[bad_row].apply(
    lambda row: ", ".join(row.index[row.to_numpy(dtype=bool)]), axis=1
)
clean = parsed.loc[~bad_row].copy()
clean = clean.astype({"order_id": "int64", "customer_id": "int64", "qty": "int64"})
if not clean["order_id"].is_unique:
    raise ValueError("Conflicting order IDs require investigation")
clean["revenue"] = clean["qty"] * clean["price"]
print(rejected[["order_id", "reason"]].to_string(index=False))
# 205 invalid_qty; 206 invalid_price; 207 invalid_date
```

The small row-wise `apply` here formats human-readable rejection reasons; it is not calculating a large numerical column. Invalid rows can have several reasons, so summing individual failure counts can exceed the count of rejected rows. Use `bad_row.sum()` to count rejected rows once.

Solution: enrich and reconcile

```
enriched = clean.merge(
    customer_lookup, how="left", on="customer_id",
    validate="many_to_one", indicator=True,
)
enriched["segment"] = enriched["segment"].astype("string").fillna("Unknown")
quality = {
    "raw_rows": len(raw_orders),
    "duplicate_rows_removed": int(duplicate_mask.sum()),
    "invalid_rows_rejected": len(rejected),
    "accepted_orders": len(enriched),
    "unmatched_customer_orders": int(enriched["_merge"].eq("left_only").sum()),
}
print(quality)
assert quality["raw_rows"] == (
    quality["duplicate_rows_removed"] + quality["invalid_rows_rejected"]
    + quality["accepted_orders"]
)
assert len(enriched) == len(clean)
np.testing.assert_allclose(enriched["revenue"].sum(), clean["revenue"].sum())
```

The reconciliation is **9 raw rows = 1 exact duplicate removed + 3 invalid rows rejected + 5 accepted orders**. One accepted order has an unmatched customer. That unmatched order is included in the accepted count, not added to it.

Solution: calculate and interpret

```
by_region = enriched.groupby("region")["revenue"].sum().sort_values(ascending=False)
by_month = enriched.set_index("date")["revenue"].resample("MS").sum(min_count=1)
average_order = enriched["revenue"].sum() / enriched["order_id"].nunique()
features = enriched[["qty", "price", "revenue"]].to_numpy(dtype=float, copy=True)
print(by_region)
print(by_month)
print("Average order value:", average_order)
print("Feature matrix shape:", features.shape)

assert set(enriched["order_id"]) == {201, 202, 203, 204, 208}
assert enriched["revenue"].sum() == 200
assert enriched["qty"].sum() == 12
assert by_month.tolist() == [140., 60.]
assert average_order == 40
```

Interpretation. The five valid orders contribute 200 currency units in revenue and 12 units sold. North contributes 110, West 60, and South 30. January contributes 140 and February 60; this tiny, partially rejected export does not establish a general downward trend. Average accepted order value is 40. Customer 99 contributes 20 and remains labelled Unknown. Rejected records must be corrected before reporting a complete business total.

Self-assessment, 20 points. Award 4 for preserving raw data and explaining the row grain; 4 for correct parsing and rejection reasons; 4 for a validated join and reconciliation; 4 for the expected metrics; and 4 for a clear interpretation that acknowledges excluded and unmatched data. Rework any stage whose checks fail, even if the final total happens to match.

STUDY CHAPTER 26

26 · Practice bank: 30 exercises

Work through these before opening Chapter 27. N means NumPy, P means pandas, and C means challenge. Allow roughly 10–15 minutes per foundation exercise and 20–30 minutes per challenge. Run the imports and `make_sales()` definition from Chapters 02 and 11 first. Each exercise supplies any additional starting data. The practice notebook includes this setup for you.

N1 · Create and inspect · Foundation

Create the integers 1 through 12 as a 3-by-4 array. Report shape, size, and column sums.

N2 · Select a rectangle · Foundation

Select rows 0 and 2 and columns 1 and 3 as a 2-by-2 rectangle. Expected result: [[1, 3], [9, 11]].

Starting data:

```
a = np.arange(12).reshape(3, 4)
```

N3 · Filter with two conditions · Foundation

Keep values that are even and greater than 5. Expected result: [6, 8, 10, 12].

Starting data:

```
a = np.arange(1, 13)
```

N4 · Protect the original · Foundation

Make an independent copy of the final two values, change its first value to 99, and prove the original stayed unchanged.

Starting data:

```
a = np.array([5, 10, 15, 20])
```

N5 · Normalise rows · Intermediate

Divide each row by its row total. Both output rows should be [0.25, 0.75]. Assume the given nonzero totals.

Starting data:

```
a = np.array([[1., 3.], [2., 6.]])
```

N6 · Column centring · Intermediate

Subtract each column mean. Verify the resulting column means are zero.

Starting data:

```
a = np.array([[2., 10.], [4., 20.], [6., 30.]])
```

N7 · Divide safely · Intermediate

Calculate numerator/denominator, using NaN for zero denominators without computing those divisions.

Starting data:

```
n = np.array([8., 6., 4.])
d = np.array([2., 0., 4.])
```

N8 · Match statistical definitions · Foundation

Calculate population and sample variance for the values. Expected results: 8/3 and 4.

Starting data:

```
a = np.array([2., 4., 6.])
```

N9 · Reproduce a random sample · Foundation

Create two independent generators with seed 123 and draw five integers in [0, 10) from each. Prove the arrays match.

N10 · Solve a system · Intermediate

Solve 2x + y = 7 and x + 3y = 11. Validate by multiplying the solution by the coefficient matrix.

Starting data:

```
A = np.array([[2., 1.], [1., 3.]])
b = np.array([7., 11.])
```

N11 · Fill by column mean · Advanced

Replace each NaN with the mean of its column and leave the input unchanged. Every column has at least one observed value.

Starting data:

```
a = np.array([[1., np.nan], [3., 4.], [np.nan, 8.]])
```

N12 · Find top values efficiently · Advanced

Return the largest three values in descending order using partitioning, then sorting only the selected values. Assume finite input and 1 <= k <= len(a).

Starting data:

```
a = np.array([8, 2, 10, 3, 7, 9])
k = 3
```

P1 · Labels and positions · Foundation

Select order 103 by label and the third row by position. Return revenue from both; both should be 90.

Starting data:

```
df = make_sales().set_index("order_id")
```

P2 · Filter orders · Foundation

Return IDs of North-region orders with revenue above 50. Expected: [103].

Starting data:

```
df = make_sales()
```

P3 · Parse and count failures · Foundation

Parse the strings as numbers. Count invalid nonmissing inputs separately from originally missing inputs. Expect one of each.

Starting data:

```
s = pd.Series(["10", "bad", None, "30"], dtype="string")
```

P4 · Normalise text · Foundation

Strip outer spaces, lowercase, and find the literal text "north". Treat missing strings as nonmatches.

Starting data:

```
s = pd.Series([" North ", "SOUTH", None], dtype="string")
```

P5 · Summarise groups · Intermediate

Calculate total revenue and distinct order count by region. Verify the regional revenue totals sum to 230.

Starting data:

```
df = make_sales()
```

P6 · Attach group context · Intermediate

Add each order's share of regional revenue while keeping six rows. Verify shares sum to 1 within every region.

Starting data:

```
df = make_sales()
```

P7 · Audit a left join · Intermediate

Left-join customer segments using many-to-one validation. Find the unmatched order ID. Expect 106.

Starting data:

```
df = make_sales()
lookup = pd.DataFrame({"customer_id": [1, 2, 3], "segment": ["R", "B", "R"]})
```

P8 · Pivot with an explicit aggregation · Intermediate

Create a region-by-product table of revenue sums. North/Book should be 90 and the grand total 230.

Starting data:

```
df = make_sales()
```

P9 · Melt a wide table · Intermediate

Reshape math and science into subject/score rows, preserving student. Expect four rows and a score sum of 340.

Starting data:

```
wide = pd.DataFrame({"student": ["A", "B"], "math": [80, 90], "science": [70, 100]})
```

P10 · Calculate monthly totals · Intermediate

Calculate month-start-labelled revenue totals. Expected values: January 140 and February 90.

Starting data:

```
df = make_sales()
```

P11 · Build a leakage-free lag · Intermediate

Sort by customer and time, then add each customer's previous revenue. First observations must be missing; customer A's later value should have lag 10.

Starting data:

```
df = pd.DataFrame({"customer": ["A", "B", "A"], "date": pd.to_datetime(["2026-01-01", "2026-01-01",
    "2026-01-02"]), "revenue": [10, 50, 20]})
```

P12 · Keep the newest record · Intermediate

Keep the latest row for each ID using the update timestamp. The two retained scores should be 20 and 30.

Starting data:

```
df = pd.DataFrame({"id": [1, 2, 1], "updated": pd.to_datetime(["2026-01-01", "2026-01-02",
    "2026-01-03"]), "score": [10, 30, 20]})
```

C1 · Diagnose silent alignment · Challenge

Predict a+b. Then explicitly reorder b to a's labels and calculate the sum. Explain why using raw values initially would be wrong.

Starting data:

```
a = pd.Series([100, 200], index=["x", "y"])
b = pd.Series([1, 2], index=["y", "x"])
```

C2 · Aggregate rates correctly · Challenge

Calculate the overall conversion rate. Compare it with the unweighted mean of the two group rates and explain the difference.

Starting data:

```
df = pd.DataFrame({"visits": [10, 1000], "conversions": [5, 100]})
```

C3 · Combine chunk statistics · Challenge

Compute the global mean from chunks without concatenating them. Explain why averaging their means gives the wrong answer.

Starting data:

```
chunks = [np.array([1., 3.]), np.array([10.])]
```

C4 · Predict broadcast shapes · Challenge

Predict the result shape for (4, 1, 3) plus (1, 5, 1). Then verify it. Explain why (4, 3) plus (4,) fails.

Starting data:

```
a = np.zeros((4, 1, 3))
b = np.ones((1, 5, 1))
```

C5 · Reuse training parameters · Challenge

Standardise the test row using training means and population standard deviations only. Expected result: [[3, 0]].

Starting data:

```
train = np.array([[0., 10.], [2., 30.]])
test = np.array([[4., 20.]])
```

C6 · Catch a duplicate lookup key · Challenge

Demonstrate that a many-to-one validated merge rejects the duplicated customer ID in lookup. Do not silently remove either conflicting segment.

Starting data:

```
df = make_sales()
lookup = pd.DataFrame({"customer_id": [1, 1, 2], "segment": ["Retail", "Business", "Business"]})
```

STUDY CHAPTER 27

27 · Solutions and reasoning

These are reference solutions, not the only valid implementations. Each solution recreates its starting data, so it does not rely on changes made in another exercise. Imports and `make_sales()` are still required. Check the output and explain the invariant before moving on.

N1 · Create and inspect · Foundation

```
a = np.arange(1, 13).reshape(3, 4)
print(a.shape, a.size, a.sum(axis=0))
np.testing.assert_array_equal(a.sum(axis=0), [15, 18, 21, 24])
```

The stop value in arange is excluded. Reducing axis 0 removes the row dimension, leaving four column totals.

N2 · Select a rectangle · Foundation

```
a = np.arange(12).reshape(3, 4)
result = a[np.ix_([0, 2], [1, 3])]
print(result)
np.testing.assert_array_equal(result, [[1, 3], [9, 11]])
```

Two same-length advanced index arrays select paired coordinates. np.ix_ requests every row/column combination.

N3 · Filter with two conditions · Foundation

```
a = np.arange(1, 13)
result = a[(a % 2 == 0) & (a > 5)]
print(result)
np.testing.assert_array_equal(result, [6, 8, 10, 12])
```

Use parenthesised comparisons and the elementwise & operator.

N4 · Protect the original · Foundation

```
a = np.array([5, 10, 15, 20])
b = a[-2:].copy()
b[0] = 99
print(a, b)
np.testing.assert_array_equal(a, [5, 10, 15, 20])
assert not np.shares_memory(a, b)
```

The slice alone would share storage. The explicit copy creates independent element storage.

N5 · Normalise rows · Intermediate

```
a = np.array([[1., 3.], [2., 6.]])
result = a / a.sum(axis=1, keepdims=True)
print(result)
np.testing.assert_allclose(result, [[0.25, 0.75], [0.25, 0.75]])
```

The row totals have shape (2, 1), so each row receives its own denominator. Without keepdims, this square example could silently divide by columns instead.

N6 · Column centring · Intermediate

```
a = np.array([[2., 10.], [4., 20.], [6., 30.]])
result = a - a.mean(axis=0, keepdims=True)
print(result)
np.testing.assert_allclose(result.mean(axis=0), 0)
```

The mean has shape (1, 2) and broadcasts over three observations. Result rows are [-2, -10], [0, 0], and [2, 10].

N7 · Divide safely · Intermediate

```
n = np.array([8., 6., 4.])
d = np.array([2., 0., 4.])
result = np.divide(n, d, out=np.full_like(n, np.nan), where=d != 0)
print(result)
np.testing.assert_allclose(result, [4, np.nan, 1], equal_nan=True)
```

The initialized output supplies the skipped NaN value. np.where would first evaluate an ordinary division passed as an argument.

N8 · Match statistical definitions · Foundation

```
a = np.array([2., 4., 6.])
population = a.var(ddof=0)
sample = a.var(ddof=1)
print(population, sample)
np.testing.assert_allclose([population, sample], [8 / 3, 4])
```

The squared deviations sum to 8. Dividing by n gives 8/3; dividing by n-1 gives 4.

N9 · Reproduce a random sample · Foundation

```
first = np.random.default_rng(123).integers(0, 10, size=5)
second = np.random.default_rng(123).integers(0, 10, size=5)
print(first, second)
np.testing.assert_array_equal(first, second)
```

Matching generator initialisation and matching call sequences reproduce the draws in the same environment. Two sequential draws from one generator advance its state.

N10 · Solve a system · Intermediate

```
A = np.array([[2., 1.], [1., 3.]])
b = np.array([7., 11.])
answer = np.linalg.solve(A, b)
print(answer)
np.testing.assert_allclose(answer, [2, 3])
np.testing.assert_allclose(A @ answer, b)
```

Use a linear solver. The solution [2, 3] satisfies both equations; elementwise multiplication would not perform this validation.

N11 · Fill by column mean · Advanced

```
a = np.array([[1., np.nan], [3., 4.], [np.nan, 8.]])
column_means = np.nanmean(a, axis=0, keepdims=True)
result = np.where(np.isnan(a), column_means, a)
print(result)
np.testing.assert_allclose(result, [[1, 6], [3, 4], [2, 8]])
assert np.isnan(a).sum() == 2
```

The missing-aware means are [2, 6]. An all-missing column would require an explicit fallback or rejection policy.

N12 · Find top values efficiently · Advanced

```
a = np.array([8, 2, 10, 3, 7, 9])
k = 3
positions = np.argpartition(a, -k)[-k:]
result = np.sort(a[positions])[::-1]
print(result)
np.testing.assert_array_equal(result, [10, 9, 8])
```

Partitioning finds the top subset without fully ordering the array. Tied cutoff values can yield different valid positions; define a tie policy when identities matter.

P1 · Labels and positions · Foundation

```
df = make_sales().set_index("order_id")
by_label = df.loc[103, "revenue"]
by_position = df.iloc[2]["revenue"]
print(by_label, by_position)
assert by_label == by_position == 90
```

The value 103 is a business-key label; 2 is a zero-based row position. The chained expression here only reads; use a single .loc operation when assigning.

P2 · Filter orders · Foundation

```
df = make_sales()
result = df.loc[(df["region"] == "North") & (df["revenue"] > 50), "order_id"]
print(result.tolist())
assert result.tolist() == [103]
```

The mask is built from the same DataFrame, preserving alignment.

P3 · Parse and count failures · Foundation

```
s = pd.Series(["10", "bad", None, "30"], dtype="string")
numbers = pd.to_numeric(s, errors="coerce")
invalid = s.notna() & numbers.isna()
print(numbers.tolist(), int(invalid.sum()), int(s.isna().sum()))
assert invalid.sum() == 1 and s.isna().sum() == 1
```

Coercion alone merges invalid and absent values into missingness. Retaining the raw input lets you distinguish the causes.

P4 · Normalise text · Foundation

```
s = pd.Series([" North ", "SOUTH", None], dtype="string")
normal = s.str.strip().str.lower()
matches = normal.str.contains("north", regex=False, na=False)
print(normal.tolist(), matches.tolist())
assert matches.tolist() == [True, False, False]
```

regex=False makes the search literal and na=False states how unknown strings affect this filter.

P5 · Summarise groups · Intermediate

```
df = make_sales()
result = df.groupby("region", as_index=False).agg(
    revenue=("revenue", "sum"), orders=("order_id", "nunique")
)
print(result)
assert result["revenue"].sum() == 230
assert result["orders"].tolist() == [2, 2, 2]
```

Aggregation creates one row per region. Distinct order count remains conceptually correct if order-level identifiers repeat across line items.

P6 · Attach group context · Intermediate

```
df = make_sales()
totals = df.groupby("region")["revenue"].transform("sum")
df["share"] = df["revenue"] / totals
print(df[["order_id", "share"]])
assert len(df) == 6
np.testing.assert_allclose(df.groupby("region")["share"].sum().to_numpy(), 1)
```

transform broadcasts each group total back to its original rows and retains their index alignment.

P7 · Audit a left join · Intermediate

```
df = make_sales()
lookup = pd.DataFrame({"customer_id": [1, 2, 3], "segment": ["R", "B", "R"]})
result = df.merge(lookup, how="left", on="customer_id", validate="many_to_one", indicator=True)
unmatched = result.loc[result["_merge"] == "left_only", "order_id"]
print(unmatched.tolist())
assert unmatched.tolist() == [106]
assert len(result) == len(df)
```

A left join preserves the order from customer 4 even though that customer is absent from the lookup. An inner join would drop it.

P8 · Pivot with an explicit aggregation · Intermediate

```
df = make_sales()
result = df.pivot_table(index="region", columns="product", values="revenue", aggfunc="sum", fill_value=0,
     observed=True)
print(result)
assert result.loc["North", "Book"] == 90
assert result.to_numpy().sum() == 230
```

The aggregation is explicitly sum; the pivot_table default mean would answer a different question.

P9 · Melt a wide table · Intermediate

```
wide = pd.DataFrame({"student": ["A", "B"], "math": [80, 90], "science": [70, 100]})
result = wide.melt(id_vars="student", var_name="subject", value_name="score")
print(result)
assert len(result) == 4 and result["score"].sum() == 340
```

Student identifies the observation; subject stores the former column labels and score stores the values.

P10 · Calculate monthly totals · Intermediate

```
df = make_sales()
result = df.set_index("date").sort_index()["revenue"].resample("MS").sum(min_count=1)
print(result)
assert result.tolist() == [140, 90]
```

MS labels bins at month starts. min_count=1 distinguishes a bin with no valid measurements from a measured zero total.

P11 · Build a leakage-free lag · Intermediate

```
df = pd.DataFrame({"customer": ["A", "B", "A"], "date": pd.to_datetime(["2026-01-01", "2026-01-01",
    "2026-01-02"]), "revenue": [10, 50, 20]})
result = df.sort_values(["customer", "date"]).copy()
result["previous"] = result.groupby("customer")["revenue"].shift(1)
print(result)
assert result.loc[2, "previous"] == 10
assert result["previous"].isna().sum() == 2
```

Grouping prevents a different customer's record from supplying the lag. Real data needs a tie policy if an entity has several rows with the same timestamp.

P12 · Keep the newest record · Intermediate

```
df = pd.DataFrame({"id": [1, 2, 1], "updated": pd.to_datetime(["2026-01-01", "2026-01-02",
    "2026-01-03"]), "score": [10, 30, 20]})
result = df.sort_values("updated").drop_duplicates("id", keep="last").sort_values("id")
print(result)
assert result["id"].is_unique
assert result["score"].tolist() == [20, 30]
```

keep="last" is meaningful only after ordering by the field that defines latest. Equal timestamps would require an additional rule.

C1 · Diagnose silent alignment · Challenge

```
a = pd.Series([100, 200], index=["x", "y"])
b = pd.Series([1, 2], index=["y", "x"])
result = a + b
reordered = b.reindex(a.index)
print(result)
print(reordered)
assert result.tolist() == [102, 201]
np.testing.assert_array_equal(a.to_numpy() + reordered.to_numpy(), result.to_numpy())
```

Pandas matches x with x and y with y. Raw input order would incorrectly produce [101, 202].

C2 · Aggregate rates correctly · Challenge

```
df = pd.DataFrame({"visits": [10, 1000], "conversions": [5, 100]})
overall = df["conversions"].sum() / df["visits"].sum()
unweighted = (df["conversions"] / df["visits"]).mean()
print(overall, unweighted)
np.testing.assert_allclose(overall, 105 / 1010)
np.testing.assert_allclose(unweighted, 0.3)
```

The overall rate is about 10.40%, whereas the mean of group rates is 30%. The overall denominator gives each visit equal weight; averaging group rates gives each group equal weight.

C3 · Combine chunk statistics · Challenge

```
chunks = [np.array([1., 3.]), np.array([10.])]
total = sum(chunk.sum() for chunk in chunks)
count = sum(chunk.size for chunk in chunks)
mean = total / count
print(mean)
np.testing.assert_allclose(mean, 14 / 3)
```

The correct mean is 14/3. Averaging chunk means would give (2 + 10)/2 = 6, over-weighting the smaller chunk.

C4 · Predict broadcast shapes · Challenge

```
a = np.zeros((4, 1, 3))
b = np.ones((1, 5, 1))
result = a + b
print(result.shape)
assert result.shape == (4, 5, 3)
try:
    np.zeros((4, 3)) + np.ones(4)
except ValueError:
    print("Expected incompatibility: trailing dimensions 3 and 4")
else:
    raise AssertionError("An incompatible shape unexpectedly succeeded")
```

Comparing dimensions from the right gives 3, then 5, then 4. For the failing pair, the final 3 and 4 are unequal and neither is 1.

C5 · Reuse training parameters · Challenge

```
train = np.array([[0., 10.], [2., 30.]])
test = np.array([[4., 20.]])
mean = train.mean(axis=0, keepdims=True)
scale = train.std(axis=0, ddof=0, keepdims=True)
result = np.divide(test - mean, scale, out=np.zeros_like(test), where=scale != 0)
print(result)
np.testing.assert_allclose(result, [[3, 0]])
```

Training means are [1, 20], and standard deviations are [1, 10]. Re-estimating on the single test row would erase the information and leak evaluation data into preprocessing.

C6 · Catch a duplicate lookup key · Challenge

```
df = make_sales()
lookup = pd.DataFrame({"customer_id": [1, 1, 2], "segment": ["Retail", "Business", "Business"]})
try:
    df.merge(lookup, on="customer_id", how="left", validate="many_to_one")
except pd.errors.MergeError:
    print("Expected rejection: customer lookup keys are not unique")
else:
    raise AssertionError("Duplicate lookup keys were not detected")
```

The relationship contract catches an ambiguous lookup before it multiplies sales rows. The conflicting source records require a business decision.

STUDY CHAPTER 28

28 · Quick reference and common mistakes

NumPy reference

- Create: `np.array(values)`, `np.arange(start, stop, step)`, `np.linspace(start, stop, num)`, `np.zeros(shape)`, `np.full(shape, value)`.
- Inspect: `a.shape`, `a.ndim`, `a.size`, `a.dtype`, `a.itemsize`, `a.nbytes`.
- Select: `a[row, col]`, `a[:, 1:3]`, `a[mask]`, `a[np.ix_(rows, cols)]`.
- Shape: `a.reshape(...)`, `a.T`, `a[:, None]`, `np.concatenate(parts, axis=...)`, `np.stack(parts, axis=...)`.
- Reduce: `a.sum(axis=...)`, `a.mean(axis=...)`, `a.std(ddof=...)`, `np.quantile(a, q)`.
- Conditions: `(a > low) & (a < high)`, `np.where(mask, yes, no)`, `np.isfinite(a)`, `np.isclose(a, b)`.
- Sort: `np.sort(a)`, `np.argsort(a)`, `np.unique(a, return_counts=True)`.
- Random: `rng = np.random.default_rng(seed)`, then `rng.integers`, `rng.normal`, `rng.choice`.
- Algebra: `A @ B`, `np.linalg.solve(A, b)`, `np.linalg.lstsq(A, b, rcond=None)`.
- Validate: `np.testing.assert_array_equal` for exact values; `assert_allclose` for approximate numerical equality.

pandas reference

- Inspect: `df.head()`, `df.info()`, `df.dtypes`, `df.describe()`, `df.isna().sum()`.
- Select: `df.loc[mask, columns]`, `df.iloc[row_positions, column_positions]`.
- Assign: `df.loc[mask, "col"] = value`, `df.assign(new_col=...)`.
- Clean: `pd.to_numeric`, `pd.to_datetime`, `.astype`, `.fillna`, `.dropna`, `.drop_duplicates`.
- Text: `.str.strip()`, `.str.lower()`, `.str.contains(pattern, regex=False, na=False)`.
- Group: `.groupby(keys).agg(...)`, `.transform("sum")`, `.size()`, `.nunique()`.
- Combine: `.merge(other, on=key, validate=...)`, `pd.concat(frames, ignore_index=True)`.
- Reshape: `.pivot_table(..., aggfunc="sum")`, `.melt(...)`, `.stack()`, `.unstack()`.
- Time: `.dt`, `.resample("MS")`, `.rolling(window)`, `.shift(1)`, `.pct_change(fill_method=None)`.
- Convert: `.to_numpy(dtype=float, copy=True)`, `pd.DataFrame(array, index=..., columns=...)`.
- Validate: `pd.testing.assert_frame_equal`, `pd.testing.assert_series_equal`.

Errors and their likely causes

“Truth value … is ambiguous.” You used `and`, `or`, or `if` with an array or Series. Decide whether you need an elementwise condition or an overall `.any()`/`.all()` test.

“Operands could not be broadcast together.” Compare shapes from right to left. An inserted length-one dimension might express the intended operation; never reshape blindly until an error disappears.

A pandas result contains unexpected missing values. Check label alignment, failed parsing, unmapped lookup values, and unmatched join keys before imputing.

Revenue increases after an enrichment join. Count duplicates in the lookup key, add `validate=`, and compare both row count and totals before and after merging.

A changed slice affects the original NumPy array. Basic slicing can share memory. Use `.copy()` when independence is required.

A pandas chained assignment has no effect. Write to the intended DataFrame in a single `.loc` assignment. See the pandas 3.0 copying rules in Chapter 13.

Standard deviations disagree across libraries. Make `ddof` and the missing-value policy explicit.

The first month has no growth rate. A change calculation needs a previous observation. Missingness is meaningful here; a fabricated zero growth rate can mislead.

STUDY CHAPTER 29

29 · Concept review with explanations

Try answering each question without running code, then check the explanation.

1. Does a one-dimensional array become a column when transposed? No. Its shape remains `(n,)`; use `a[:, None]` for `(n, 1)`.

2. What does `sum(axis=0)` do to a `(5, 3)` array? It removes the first axis and returns three column totals. With `keepdims=True`, its shape is `(1, 3)`.

3. Why is a Python list useful even if arrays can be faster? Lists handle general Python objects and dynamic collections well. NumPy's advantages are strongest when work can be expressed as substantial homogeneous array operations.

4. How do `*` and `@` differ? `*` performs elementwise multiplication with broadcasting; `@` follows matrix multiplication rules, including appropriate batch behavior for higher dimensions.

5. What is the difference between `loc[1]` and `iloc[1]`? The first requests label 1. The second requests the second row. These coincide only under a suitable index.

6. Why do Series sometimes produce surprising arithmetic? They align by index label before calculating. Physical order alone does not determine matches.

7. When do you use `transform` instead of `agg`? When the result must align with the original rows, such as attaching each row's group mean. Aggregation creates group-level summaries.

8. Why validate a merge that runs without an error? A technically valid many-to-many join can multiply rows and amounts. A declared cardinality contract exposes unexpected duplicate keys.

9. Are `pivot` and `pivot_table` interchangeable? No. `pivot` requires unique index/column pairs; `pivot_table` aggregates repeated pairs according to its aggregation function.

10. Should all missing numbers be filled with zero? Only when zero correctly describes the absent observation. Unknown price, no recorded transaction, and a measured zero price are different situations.

11. Does `np.vectorize` guarantee a speed-up? No. It provides convenient array-like calling semantics, typically wrapping Python calls. Use genuine array operations or a measured alternative when speed matters.

12. Why should preprocessing parameters be fitted on training data only? Letting test data influence imputation, scaling, or category boundaries can make evaluation unrealistically optimistic.

13. How is rolling by three rows different from rolling by three days? The former counts observations; the latter uses a time interval and can include different numbers of rows when sampling is irregular.

14. Does Copy-on-Write make data-quality validation unnecessary? No. It governs mutation behavior, not the validity of calculations, keys, units, or business assumptions.

15. What is the most useful question before counting rows? “What does one row represent?” The answer determines whether row count means customers, orders, items, events, or something else.

STUDY CHAPTER 30

30 · Further projects, mastery checklist, and sources

Project A: student assessment analysis

Generate or enter fictional results for 30 students across three subjects and two terms. Store one row per student/subject/term, with a unique composite key. Include several missing scores and one exact duplicate. Validate score bounds of 0–100, preserve absent scores as unknown, calculate subject summaries, and pivot term results for comparison. Use NumPy to calculate per-subject standardised values with an explicit zero-variance policy.

Success criteria: unique composite keys after cleaning; unchanged observed-score totals after reshaping; correct handling of students missing one term; no ranking based on silently imputed scores; a written explanation of whether comparisons use the same students in both terms.

Project B: inventory movement analysis

Create a product lookup and a movement log with timestamp, product ID, and signed quantity. Positive movements add stock; negative movements remove stock. Validate lookup uniqueness, sort events deterministically, calculate cumulative stock within each product, and flag negative balances. Aggregate daily movement, then compare row-based and time-based rolling averages.

Success criteria: opening stock plus total movements equals closing stock for every product; unmatched product IDs are reported; missing event times are isolated; joins preserve movement totals; no cumulative calculation crosses product boundaries. State whether no movement on a day means zero or unavailable information.

Ready-to-use mastery checklist

- I can predict an array's shape before executing a reshape, reduction, or broadcast.
- I can explain when NumPy operations share memory and when they copy.
- I can choose `ddof`, weights, and a missing-value policy deliberately.
- I can select and assign with `.loc` and `.iloc` without guessing.
- I can diagnose index alignment and preserve meaningful labels during conversion.
- I can distinguish row count, non-null count, and distinct-key count.
- I can choose aggregation, transformation, or filtering by required output shape.
- I can validate join cardinality and reconcile row counts and totals.
- I can parse dates, sort time series, and distinguish resampling, rolling, and lagging.
- I can preserve raw data, document rejected rows, and reproduce an analysis from the first cell.
- I can complete at least 24 of 30 exercises without copying a solution, then redo the remaining exercises after review.
- I can complete the capstone and explain its caveats in plain language.

Official references and version notes

The explanations and datasets here are original teaching material. These primary sources were used to check selected API details and version-sensitive behavior on 14 September 2026. Stable documentation links may point to newer patch versions later. Consult documentation matching your installed version when behavior differs.

- NumPy beginner guide: array concepts and basic operations.
- NumPy copies and views: storage sharing and indexing behavior.
- NumPy broadcasting: dimensional compatibility.
- NumPy random Generator: generator-based sampling and compatibility notes.
- NumPy standard deviation and pandas Series standard deviation: `ddof` defaults.
- NumPy linear solver: square-system requirements.
- pandas introduction: core structures and workflows.
- pandas Copy-on-Write: pandas 3.0 mutation rules.
- pandas string migration guide: pandas 3.0 string inference.
- pandas missing data: missing-value and nullable-dtype behavior.
- pandas GroupBy: grouped operations.
- pandas merging and merge API: join cardinality and null-key behavior.
- pandas pivot table API: aggregation and categorical options.
- pandas time series: datetime, time zone, frequency, and window concepts.

Core examples and all 30 exercise solutions were executed with Python 3.12.14, NumPy 2.3.5, and pandas 2.2.3. The pandas 3.0 notes were checked against official documentation. Optional external-file, installation, and plotting recipes were not included in that execution count.